

Notes

Alex Loomis

November 11, 2024

1 Statement of Problem

Consider a collection of units on a (connected component of a) grid, along with a collection of (at least as many) targets. We wish to find a collection of paths for the units to the targets that

- minimizes the makespan (the maximum time to target),
- involves no unit conflicts (collisions).

In our setting, the units are rectangular, and all of the same size. The units move at discrete time steps, indexed by $\mathbb{N}$. Each time step, each unit may either wait at its current location, or move one cell in any cardinal direction, subject to the constraint to remain on the grid.

We will take the upper-leftmost cell that a unit occupies to be its origin. Each grid cell has a (strictly) positive integer cost to enter, and a unit whose origin enters a cell must wait that many time steps before moving again.

For example, in the following diagram...

2 Overview of Approach

To find these paths, we will begin by finding individual paths for each unit, then will iteratively improve upon those paths. This has the same structure as an A* search.

```
1 fn cbs(spec: Specification) -> HashMap<Path> {
2   let mut open = Vec::new();
3   let mut init = HashMap::new();
4   for unit in spec.units {
5     let path = find_path(unit, spec.targets);
6     init.insert(unit, path);
7   }
8   open.push(make_node(init));
9   loop {
10    let node = choose_node(open);
11    if node.conflicts.is_empty() {
12      return node.paths;
13    }
14    for child in node.expand_node() {
15      open.push(child);
16    }
17  }
18 }
```

The biggest determinants of how many loops will be required are what nodes we return from `choose_node` and from `expand_node`. In general, we want these functions to return nodes that are close to being correct. We will follow a conflict-based search (CBS) approach. In CBS, we add constraints to eliminate conflicts, until we have a conflict-free collection of paths.

3 MDDs

We will represent unit paths by multi-decision diagrams (MDDs). These are ... We will use them to find sets of mutually exclusive states.

4 Mutexes

Our nodes consist of the following information:

- a set of constraints,
- a minimal length MDD for each unit,
- the length of the longest MDD.

References

References

- [1] Ariel Felner et al. "Adding Heuristics to Conflict-Based Search for Multi-Agent Path Finding". In: *Proceedings of the International Conference on Automated Planning and Scheduling* 28 (June 2018), pp. 83–87. DOI: 10.1609/icaps.v28i1.13883.
- [2] Jiaoyang Li et al. "Disjoint Splitting for Multi-Agent Path Finding with Conflict-Based Search". In: *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. 2019, pp. 279–283.
- [3] Guni Sharon et al. "Conflict-based search for optimal multi-agent pathfinding". In: *Artificial Intelligence* 219 (2015), pp. 40–66. ISSN: 0004-3702. DOI: <https://doi.org/10.1016/j.artint.2014.11.006>. URL: <https://www.sciencedirect.com/science/article/pii/S0004370214001386>.
- [4] Han Zhang et al. "Multi-agent path finding with mutex propagation". In: *Artificial Intelligence* 311 (2022), p. 103766. ISSN: 0004-3702. DOI: <https://doi.org/10.1016/j.artint.2022.103766>. URL: <https://www.sciencedirect.com/science/article/pii/S0004370222001060>.
- [5] Han Zhang et al. "Mutex Propagation in Multi-Agent Path Finding for Large Agents". In: *Proceedings of the International Symposium on Combinatorial Search* 15 (July 2022), pp. 249–253. DOI: 10.1609/socs.v15i1.21776.